Résolution des systèmes linéaires

1 Présentation

La résolution de systèmes linéaires est rencontrée dans de très nombreuses applications scientifiques. On veut ici résoudre un système du type

$$A \underbrace{x}_{inconnue} = \underbrace{b}_{second\ membre}, \tag{1}$$

avec $A \in \mathbb{R}^{n\times n}$ non singulière (inversible), $x \in \mathbb{R}^n$, $b \in \mathbb{R}^n$. Les résultats de ce chapitre peuvent également s'appliquer pour $A \in \mathbb{C}^{n\times n}$.

Les méthodes les plus couramment utilisées consistent à transformer le système initial (1) en des systèmes plus faciles à résoudre en décomposant (ou factorisant) A comme produit de matrices triangulaires. Ces méthodes sont appelées *méthodes directes* et reposent pour la plupart sur l'élimination de Gauss. S'il n'y a pas d'erreur d'arrondi, ces méthodes permettent d'obtenir la solution "exacte" de (1) en un nombre fini d'étapes.

D'autres méthodes dites *itératives* consistent à générer une suite de vecteurs $x^{(0)}, x^{(1)}, x^{(2)}, \ldots$ qui converge vers une approximation de la solution du système (1).

Le choix entre les deux types de méthodes se fera en fonction de la connaissance que l'on a de A (taille, structure...), de la précision demandée, du temps autorisé pour le calcul et de la mémoire disponible.

2 Méthodes directes

2.1 Systèmes triangulaires

On résout $Lx = b$ avec $L = (l_{ij})$ **triangulaire inférieure** et non singulière, i.e:

$$l_{ij} \;=\; 0 \text{ si } j > i \text{ et } \forall i,\; l_{ii} \;\neq\; 0.$$

Le système linéaire s'écrit alors

$$\begin{cases} l_{11}\, x_1 & & & = b_1 \\ l_{21}\, x_1 + l_{22}\, x_2 & & & = b_2 \\ \quad\vdots & & & \quad\vdots \\ l_{n1}\, x_1 + l_{n2}\, x_2 & + \ldots & +l_{nn}\, x_n & = b_n \end{cases} \tag{2}$$

Compte tenu de sa forme particulière, un tel système se résout de proche en proche en calculant x_1 à partir de l'équation 1, puis x_2 à partir de l'équation 2 après avoir substitué la valeur de x_1, etc. Cela donne

$$x_1 = b_1/l_{11},$$

puis

$$x_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} x_j \right). \tag{3}$$

Comme on calcule successivement x_1, x_2, ..., on parle d'algorithme de *substitution avant* ou *d'algorithme de descente*.

Le **coût calculatoire** (ou complexité) d'un algorithme numérique est obtenu en calculant le nombre total d'opérations arithmétiques nécessaires $(+, -, \times, \div)$. On parle alors de *flops* (nom abrégé pour *floating-point operations*).
Exercice: Montrer que la résolution du système (2) nécessite n^2 flops (indice: compter séparément les opérations $+, -$ puis $\times, \div$ et additionner).

Remarques:

- On accède à L par ligne, ce qui potentiellement aura un impact sur la performance selon si les données sont contigües en mémoire ou pas (risque de défaut de cache). Cela dépendra du langage de programmation utilisé (par exemple en C les tableaux sont stockés ligne par ligne alors qu'en Fortran ils sont stockés colonne par colonne).
- Le calcul de x_i par la formule (3) ne nécessite que de connaître b_i et les x_j précédents $(j < i)$. On peut donc stocker directement x dans b, ce qui permet d'économiser de la mémoire.

Dans le cas d'une matrice **triangulaire supérieure**, on résout $Ux = b$ avec $U = (u_{ij})$ telle que

$$u_{ij} \;=\; 0 \text{ si } i > j \text{ et } \forall i,\; u_{ii} \;\neq\; 0.$$

La résolution se fait alors de proche en proche en calculant x_n, x_{n-1}, etc. Cela donne

$$x_n = b_n / u_{nn},$$

puis

$$x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^{n} u_{ij} x_j \right).$$

On parle alors d'algorithme de *substitution arrière* ou *d'algorithme de remontée*, qui nécessite également n^2 flops.

2.2 Systèmes linéaires généraux: factorisation LU

2.2.1 Principe

La factorisation LU consiste à décomposer une matrice carrée inversible A en un produit $A = L\,U$ avec

- L matrice triangulaire inférieure avec des 1 sur la diagonale,
- U matrice triangulaire supérieure.

On a alors $Ax = b \;\Leftrightarrow\; L\,\underbrace{U\,x}_{y} = b$, et donc le système linéaire $Ax = b$ se résout en résolvant successivement les 2 systèmes triangulaires: $Ly = b$, puis $Ux = y$.

La factorisation LU est obtenue par la méthode bien connue de l'élimination de Gauss qui consiste à transformer le système linéaire en un système triangulaire en éliminant les inconnues en dessous de la diagonale par des combinaisons linéaires adéquates des équations (voir paragraphe suivant).

2.2.2 Exemple et algorithme

Nous allons illustrer la méthode d'obtention de L et U sur la matrice $A = \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 10 \end{pmatrix}$.
La méthode de l'élimination de Gauss consiste à mettre à 0 les termes sous la diagonale en procédant colonne par colonne. Pour cela on remplace chaque ligne sous la diagonale par une combinaison linéaire de cette ligne et de la ligne contenant l'élément diagonal. Par exemple pour la matrice A, on commence par mettre à 0 le terme a_{21} en remplaçant la deuxième ligne L_2 par $L_2 - \frac{a_{21}}{a_{11}} L_1 = L_2 - 2\, L_1$ (le terme a_{11} est appelé pivot). De même, pour mettre à 0 le terme a_{31}, on effectue $L_3 \leftarrow L_3 - \frac{a_{31}}{a_{11}} L_1 = L_3 - 3\, L_1$. On obtient donc:

$$A = \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 10 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 4 & 7 \\ 0 & -3 & -6 \\ 0 & -6 & -11 \end{pmatrix}.$$

On continue avec la deuxième colonne en mettant à zéro a_{32} par la substitution $L_3 \leftarrow L_3 - \frac{a_{32}}{a_{22}} L_2 = L_3 - 2\, L_2$ (a_{22} est le pivot) pour obtenir ainsi U:

$$\begin{pmatrix} 1 & 4 & 7 \\ 0 & -3 & -6 \\ 0 & -6 & -11 \end{pmatrix} \longrightarrow U = \begin{pmatrix} 1 & 4 & 7 \\ 0 & -3 & -6 \\ 0 & 0 & 1 \end{pmatrix}.$$

La matrice L est la matrice triangulaire inférieure avec des 1 sur la diagonale et les coefficients sous la diagonale correspondent aux différents coefficients multiplicateurs utilisés à chaque élimination ($\frac{a_{ij}}{a_{jj}}$, $i > j$, en rouge précédemment)[1]. On a donc:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 2 & 1 \end{pmatrix}.$$

Pour une matrice $n \times n$, la factorisation LU s'obtient par l'algorithme ci-dessous:

Algorithm 2.1 Factorisation LU "in place" sans pivotage (L et U sont stockés dans A)

```
L est stocké dans le triangle inférieur hors diagonale de A (la diagonale de L, faite de 1,
n'est pas stockée), U est stocké dans le triangle supérieur (y compris diagonale) de A.
for j = 1 : n − 1 do                          ▷ j = indice de colonne
    for i = j + 1 : n do                      ▷ lignes sous la diagonale
        a_ij = a_ij / a_jj                    ▷ a_ij contient l_ij, le multiplicateur de Gauss
        for k = j + 1 : n do                  ▷ mise à jour du reste de la ligne i
            a_ik = a_ik − a_ij × a_jk         ▷ a_ik contient u_ik
        end for
    end for
end for
```

Cet algorithme nécessite $2n^3/3$ flops. En pratique, afin d'optimiser l'occupation mémoire, les facteurs U et L sont stockés dans A, évitant ainsi d'utiliser 2 matrices supplémentaires i.e $a_{ij} = u_{ij}$ si $i \leq j$ et $a_{ij} = l_{ij}$ si $i > j$ (inutile de stocker les 1). Si la matrice de départ A doit être ré-utilisée plus tard dans les calculs, il suffit d'en garder une copie avant factorisation.

[1] Pour plus de détails, voir [G. H. Golub and C. F. Van Loan, Matrix Computations, The Johns Hopkins University Press, 4th edition, 2013] .

2.2.3 Pivotage par permutation des lignes

En pratique, on peut avoir des problèmes dans l'élimination de Gauss lorsque:

- $a_{jj} = 0 \Rightarrow$ division par 0, par exemple si $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$,
- a_{jj} très petit $\Rightarrow$ difficultés numériques.

Pour éviter cela, on permute (en même temps) les lignes de A et de b de sorte que le plus grand nombre (en valeur absolue) de la colonne courante j, i.e $\max_{i \geq j} |a_{ij}|$ se trouve en position diagonale et devienne donc le pivot. Si ce terme se trouve sur la ligne numéro k, alors on permute les lignes j et k.
La factorisation obtenue à la fin de l'algorithme s'écrira

$$PA = LU,$$

où P est une matrice de permutation[2]. On a alors:

$$Ax = b \;\Leftrightarrow\; PA\,x = Pb \;\Leftrightarrow\; L\underbrace{Ux}_{y} \;=\; Pb,$$

et le système linéaire se résout en résolvant successivement les systèmes triangulaires $Ly = Pb$, puis $Ux = y$.

Prenons l'exemple de la matrice $A = \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 10 \end{pmatrix}$ vue dans le paragraphe précédent.

Nous commençons ici par la première colonne et observons que 3 est le nombre le plus grand. Nous permutons donc les lignes 1 et 3. Cependant il convient de garder la trace de cette permutation en indiquant dans un vecteur auxiliaire p les numéros de ligne avant les permutations. D'un point de vue pratique, on choisit de positionner p à droite de la matrice A. Cela nous donne, après permutation des lignes 1 et 3:

$$[A|p] = \begin{pmatrix} 3 & 6 & 10 & | & 3 \\ 2 & 5 & 8 & | & 2 \\ 1 & 4 & 7 & | & 1 \end{pmatrix}.$$

On procède alors à l'élimination de Gauss pour la première colonne en mettant à 0 les termes sous la diagonale. Pour mettre à 0 les termes a_{21} et a_{31}, on effectue respectivement $L_2 \leftarrow L_2 - 2/3\ L_1$ et $L_3 \leftarrow L_3 - 1/3\ L_1$. Cependant au lieu d'écrire 0 à la place des termes a_{21} et a_{31}, on remplace ces derniers par les valeurs des coefficients 2/3 et 1/3 (en rouge ci-dessous) qui seront les termes du facteur L. Cela permet d'une part de stocker U et la partie sous-diagonale de L (la diagonale de L étant nécessairement avec des 1) dans la même matrice pour économiser de la mémoire mais également de *permuter en même temps les valeurs de* L lors des prochaines permutations de lignes. On obtient ainsi:

$$[A|p] = \begin{pmatrix} 3 & 6 & 10 & | & 3 \\ 2/3 & 1 & 4/3 & | & 2 \\ 1/3 & 2 & 11/3 & | & 1 \end{pmatrix}.$$

On procède de même pour la deuxième colonne en mettant 2 en position diagonale par permutation des lignes 2 et 3 (mais bien sûr les numéros de lignes qui sont mis dans p sont

[2]Une matrice de permutation est une matrice qui contient un seul 1 dans chaque ligne et dans chaque colonne, et des 0 partout ailleurs. PA permute les lignes de A et AP permute les colonnes de A.

les numéros d'origine, avant le début de la factorisation) puis on procède à l'élimination de Gauss, ce qui donne:

$$[A|p] = \begin{pmatrix} 3 & 6 & 10 & | & 3 \\ 1/3 & 2 & 11/3 & | & 1 \\ 2/3 & 1 & 4/3 & | & 2 \end{pmatrix} \longrightarrow \begin{pmatrix} 3 & 6 & 10 & | & 3 \\ 1/3 & 2 & 11/3 & | & 1 \\ 2/3 & 1/2 & -1/2 & | & 2 \end{pmatrix}.$$

La matrice U correspond à la partie triangulaire supérieure (y compris la diagonale) de la matrice ci-dessus. La matrice L est alors la partie sous-diagonale à laquelle on rajoute des 1 sur la diagonale. La matrice de permutation P (qui est une permutation de la matrice identité) s'obtient à partir du vecteur p en mettant, pour chaque ligne i, 1 sur P_{ij} si $p_i = j$ et 0 ailleurs. Nous obtenons donc:

$$U = \begin{pmatrix} 3 & 6 & 10 \\ 0 & 2 & 11/3 \\ 0 & 0 & -1/2 \end{pmatrix}, L = \begin{pmatrix} 1 & 0 & 0 \\ 1/3 & 1 & 0 \\ 2/3 & 1/2 & 1 \end{pmatrix}, P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

En comparant avec le résultat de la section 2.2.2 on observe qu'avec pivotage on n'obtient pas la même factorisation que sans pivotage.

Le coût calculatoire est toujours de $2n^3/3$ flops que l'on permute les lignes ou pas. Cependant, avec le pivotage, on rajoute un surcoût de $\mathcal{O}(n^2)$ comparaisons pour trouver les nouveaux pivots. Cela n'est pas négligeable en temps d'exécution lorsque n est grand car cela entraine des mouvements de données supplémentaires. En effet, sur les ordinateurs récents, le temps pour une communication est beaucoup plus élevé que celui d'une opération arithmétique (on peut faire des milliers d'opérations arithmétiques pour le coût d'un message).

3 Cas des matrices symétriques définies positives

Les matrices symétriques définies positives, en anglais *symmetric positive definite* (SPD), forment une classe de matrices souvent rencontrée dans la résolution d'équations différentielles ou de problèmes d'optimisation. Elle permettent un stockage mémoire moindre (seulement la partie triangulaire inférieure ou supérieure de la matrice, soit $n(n+1)/2$ éléments, et une factorisation "symétrique" plus économique en nombre d'opérations.

Déf 1. *$A \in \mathbb{R}^{n\times n}$ symétrique est dite définie positive ssi $\forall x \in \mathbb{R}^n$ $(x \neq 0)$, $x^T Ax > 0$.*

Prop 1. *Si $A \in \mathbb{R}^{n\times n}$ est définie positive alors ses éléments diagonaux sont strictement positifs.* A vérifier en exercice.

Pour les matrices SPD, on utilise la *factorisation de Cholesky* qui consiste à décomposer A en facteurs triangulaires tout en exploitant et conservant la symétrie.

La factorisation d'une matrice A SPD donne $A = LL^T$ avec L triangulaire inférieure. On a alors:

$Ax = b \quad \Leftrightarrow \quad L \underbrace{L^T x}_{y} = b$, et donc le système linéaire $Ax = b$ se résout en résolvant successivement les 2 systèmes triangulaires: $Ly = b$, puis $L^T x = y$.

Remarques:

- Il n'est pas nécessaire de pivoter dans la factorisation de Cholesky[3].
- Il y a existence et unicité de la décomposition de Cholesky pour toute matrice SPD.
- Une autre formulation est $A = U^T U$ avec U triangulaire supérieure.
- Si l'algorithme de Cholesky va jusqu'au bout, alors la matrice est SPD.

La méthode pour obtenir la factorisation $A = LL^T$ d'une matrice SPD consiste à partitionner et décomposer A sous la forme

$$A = \begin{pmatrix} a_{11} & c^T \\ c & A_{n-1} \end{pmatrix} = \begin{pmatrix} \sqrt{a_{11}} & 0 \\ \frac{c}{\sqrt{a_{11}}} & I_{n-1} \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & A_{n-1} - \frac{c\,c^T}{a_{11}} \end{pmatrix} \begin{pmatrix} \sqrt{a_{11}} & \frac{c^T}{\sqrt{a_{11}}} \\ 0 & I_{n-1} \end{pmatrix} \tag{4}$$

où $a_{11} > 0$, $c \in \mathbb{R}^{(n-1)}$, $A_{n-1} \in \mathbb{R}^{(n-1)\times(n-1)}$.
Ensuite on applique la même décomposition à la matrice $S = A_{n-1} - \frac{c\,c^T}{a_{11}}$ (appelée complément de Schur et qui est également SPD) et ainsi de suite jusqu'à ce que S soit réduite à un réel.

Exemple:

$$\begin{aligned} \begin{pmatrix} 2 & -2 \\ -2 & 5 \end{pmatrix} &= \begin{pmatrix} \sqrt{2} & 0 \\ -\sqrt{2} & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 5-2 \end{pmatrix} \begin{pmatrix} \sqrt{2} & -\sqrt{2} \\ 0 & 1 \end{pmatrix} \\ &= \underbrace{\begin{pmatrix} \sqrt{2} & 0 \\ -\sqrt{2} & \sqrt{3} \end{pmatrix}}_{L} \begin{pmatrix} \sqrt{2} & -\sqrt{2} \\ 0 & \sqrt{3} \end{pmatrix}. \end{aligned}$$

L'algorithme 3.1 calcule L dans le triangle inférieur de A. Il nécessite $n^3/3$ flops.

Algorithm 3.1 Factorisation de Cholesky

for $j = 1 : n$ **do**
  $a_{jj} = \sqrt{a_{jj}}$
  $a_{j+1:n,j} = a_{j+1:n,j}/a_{jj}$ ▷ Mise à jour vecteur colonne c: $c \leftarrow c/\sqrt{a_{jj}}$
  $a_{j+1:n,j+1:n} = a_{j+1:n,j+1:n} - a_{j+1:n,j}a^T_{j+1:n,j}$ ▷ Mise à jour A_{n-j} (outer product)
end for

4 Méthodes itératives

4.1 Principe

Lorsque n est grand, les méthodes directes peuvent s'avérer trop coûteuses en nombre d'opérations (en général $\mathcal{O}(n^3)$) mais aussi en occupation mémoire, en particulier lorsque A est de grande taille et creuse (i.e contient beaucoup de zéros). En effet, même s'il est possible d'optimiser le stockage des matrices creuses en évitant de stocker les zéros, les facteurs L et U obtenus peuvent malgré tout être denses et nécessiter trop de place mémoire. L'idée des méthodes itératives pour résoudre $Ax = b$ est la suivante:

- On génère une suite $(x^{(k)})$ de "solutions approchées" qui converge vers la solution x.

[3] car toute matrice SPD A est diagonale dominante i.e, $\forall i\ |a_{ii}| > \sum_{j\neq i} |a_{ij}|$

- On utilise A seulement pour des produits matrice-vecteur.
- L'efficacité de la méthode sera mesurée par la vitesse de convergence.

4.2 Méthode

On décompose la matrice A (*splitting* de A) sous la forme $A = M - N$ où M et N sont des matrices à choisir (M doit être inversible). On a alors:

$$Ax = b \;\Leftrightarrow\; (M-N)x = b \;\Leftrightarrow\; Mx = Nx + b.$$

On se donne $x^{(0)} \in \mathbb{R}^n$ et on définit alors le schéma itératif par

$$Mx^{(k+1)} = Nx^{(k)} + b. \tag{5}$$

Puisque $\lim_{k\to\infty} x^{(k)} = x$, on observe que l'équation (5) devient, par passage à la limite, $Ax = b$.
L'équation (5) s'écrit aussi $x^{(k+1)} = \underbrace{M^{-1}N}_{B} x^{(k)} + M^{-1}b$. La matrice $B = M^{-1}N$ est appelée **matrice d'itération**.

Remarques:

- $x^{(k)}$ étant calculé, on calcule alors $x^{(k+1)}$ en résolvant le système linéaire $Mx = c$ avec $c = Nx^{(k)} + b$. Il conviendra alors de choisir M telle que ce système linéaire soit facile à résoudre (par exemple diagonal ou triangulaire).
- On a $M^{-1}N = M^{-1}(M - A) = I - M^{-1}A$ donc si M est proche de A alors $M^{-1}A$ est proche de I et la convergence sera plus rapide.
- Le critère d'arrêt sera en général du type $\|x^{(k+1)} - x^{(k)}\| < \epsilon$ où ϵ est la tolérance fixée par l'utilisateur et $\|\cdot\|$ représente une norme de $\mathbb{R}^n$.

4.3 Exemples

On décompose A sous la forme $A = E + F + D$ avec E triangulaire inférieure avec des 0 sur la diagonale, F triangulaire supérieure avec des 0 sur la diagonale, D matrice diagonale (diagonale de A). On a donc

$$A = \underbrace{\begin{pmatrix} 0 & \dots & \dots & 0 \\ a_{21} & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ a_{n1} & \dots & a_{n(n-1)} & 0 \end{pmatrix}}_{E} + \underbrace{\begin{pmatrix} 0 & a_{12} & \dots & a_{1n} \\ \vdots & \ddots & & \vdots \\ \vdots & & \ddots & a_{(n-1)n} \\ 0 & \dots & \dots & 0 \end{pmatrix}}_{F} + \underbrace{\begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & \ddots & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \dots & 0 & a_{nn} \end{pmatrix}}_{D}.$$

Dans ce qui suit on suppose que A a des coefficients diagonaux non nuls.

Méthode de Jacobi: On prend $M = D$ et $N = -(E + F)$.
D'après (5) on résout à chaque itération le système diagonal

$$Dx^{(k+1)} = b - (E + F)x^{(k)}.$$

Les composants du vecteur calculé à l'itération $k + 1$ sont donc:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Méthode de Gauss-Seidel: On prend $M = D + E$ et $N = -F$.
Chaque itération consiste alors à résoudre le système triangulaire inférieur

$$(D + E)x^{(k+1)} = b - Fx^{(k)}.$$

Les composants du vecteur calculé à l'itération $k + 1$ sont alors:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right).$$

On observe que pour calculer $x_i^{(k+1)}$ on n'utilise que les composants d'indice j de $x^{(k)}$ tels que $j > i$ (contrairement à Jacobi) et les composants précédents de $x_i^{(k+1)}$. On peut donc stocker $x^{(k+1)}$ et $x^{(k)}$ dans le même vecteur.

Convergence:

- Si A est diagonale dominante alors les méthodes de Jacobi et de Gauss-Seidel convergent.
- Si A est SPD alors la méthode de Gauss-Seidel converge quelque soit $x^{(0)}$.

5 Exercices

Exercice 1

Soit la matrice $A = \begin{pmatrix} \alpha & 3 & 0 \\ 1 & \alpha & 2 \\ 0 & 1 & \alpha \end{pmatrix}$.

Indiquer les valeurs de α qu'il faut exclure pour que la factorisation (sans pivoter) de A aille à son terme.

Pour α différent de ces valeurs, donner les facteurs L et U obtenus.

Exercice 2

Calculer la factorisation LU avec pivotage de la matrice $A = \begin{pmatrix} 1 & 2 & -3 & 4 \\ 4 & 8 & 12 & -8 \\ 2 & 3 & 2 & 1 \\ -3 & -1 & 1 & -4 \end{pmatrix}$.

Exercice 3

Montrer que l'égalité (4) exprimant la première étape de la décomposition de Cholesky s'obtient par 2 éliminations de Gauss qui consistent à mettre à 0 la première colonne et la première ligne de A.

Exercice 4

Soit $A = \begin{pmatrix} 3 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 3 \end{pmatrix}$ et $b \in \mathbb{R}^3$.

Montrer que la méthode de Jacobi converge pour résoudre $Ax = b$ et exprimer la matrice d'itération.